GPU Programming Course Project

Fused Attention with cuTile

Nishitha D and Harshith M R

Abstract

Computing causal scaled dot-product attention directly materializes an $N \times N$ score matrix and a probability matrix in global memory, so its memory footprint and traffic grow quadratically with sequence length. We implement the forward pass as a single fused NVIDIA cuTile kernel that processes queries and keys in 64×64 tiles, maintains a numerically stable online softmax in FP32, skips key/value tiles above the causal diagonal, and never writes the score or probability matrix to global memory. The kernel passes all 84 correctness cases against an FP32 reference, with every output element within the tolerance budget, including inputs whose scores would overflow a naive exponential. On an NVIDIA RTX PRO 6000 Blackwell GPU it is on average $4.3\times$ faster than an unfused cuTile baseline (up to $13\times$ at $N = 8192$), $1.33\times$ faster than PyTorch SDPA and $1.32\times$ faster than NVIDIA's TileGym cuTile attention kernel, reaching 341 TFLOP/s. Three profiler-guided optimizations (heavy-first scheduling, base-2 exponentials, and halving register usage to fit two blocks per SM) account for a $1.32\times$ gain over our first fused kernel. We further extend the kernel to grouped-query attention and implement a recompute-based backward pass integrated with PyTorch autograd. Our code is available at github.com/nvidiacutile/fused-attention.

1. INTRODUCTION

Scaled dot-product attention [2] is the central operation of Transformer models. For one attention head with query, key and value matrices $Q, K, V \in \mathbb{R}^{N \times D}$ it computes

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{D}} + M\right)V, \tag{1}$$

where in causal attention the mask M removes every key that comes after the query. A direct implementation forms the score matrix $S = QK^\top/\sqrt{D}$ and the probability matrix $P = \mathrm{softmax}(S)$ as full $N \times N$ tensors in global memory and then multiplies P by V. The storage and memory traffic of these intermediates grow as N^2, while the output grows only as N. In the largest configuration we benchmark ($B = 2$ sequences, $H = 8$ heads, $N = 8192$), S in FP32 occupies 4 GiB and P in FP16 a further 2 GiB, and each is written to global memory and read back. Our unfused cuTile baseline, which follows this pattern, reaches only 23–45 TFLOP/s at $N = 8192$, about an order of magnitude below the fused kernel presented here.

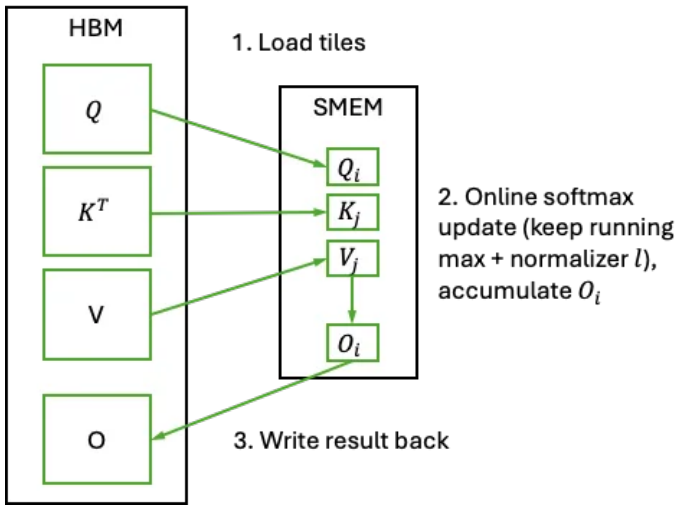


Figure 1. Tiled Fused Attention

FlashAttention [4, 5] removes these intermediates by tiling. Each block of queries streams over blocks of keys and values, keeps its scores in on-chip storage, and combines the tiles with an online softmax [3] that maintains a running row maximum and a running normalization sum, so S and P never reach global memory. Under a causal mask roughly half of the key/value tiles contribute nothing and a tiled kernel can skip them entirely.

We implement this approach in cuTile [6], NVIDIA's tile-based Python programming model. In cuTile the programmer writes operations on whole tiles, such as loads, matrix multiplications and reductions, and the compiler maps them onto threads, shared memory and tensor cores. This raises the question this report answers: how close a short tile-level kernel can come to tuned library implementations, and which optimizations remain under the programmer's control when the compiler manages the low-level hardware details. The inputs supported by our kernel are summarized in Table 1.

Table 1. Inputs supported by the fused forward kernel.

Item	Requirement
Tensors	Q, K, V: contiguous, shape $[B, H, N, D]$, `float16`
Batch and heads	$1 \le B \le 16$, $1 \le H \le 32$
Sequence length	$1 \le N \le 8192$, divisible by the query and key tile sizes (default 64×64)
Head dimension	$D \in \{32, 64, 128\}$, value head dimension equal to D
Attention	causal self-attention, $N_q = N_k = N$
Precision	FP16 inputs with FP32 accumulation and softmax statistics
Output	O: shape $[B, H, N, D]$, `float16`
Interface	`python run_attention.py --input X.npz --output O.npy --backend cutile`

Section 2 derives the tiled algorithm and the online-softmax recurrence on which the kernel is built, and Section 3 shows that it matches an FP32 reference on every test case. Sections 4 and 5 describe how we measure performance and compare the fused kernel against the unfused baseline, PyTorch SDPA and NVIDIA's TileGym kernel. Section 6 follows the profiler-guided changes that took the first fused kernel to its final form, including those that did not help. Sections 7 and 8 carry the design over to grouped-query attention and to the backward pass, and Section 9 closes with the kernel's limitations and directions for future work.

2. ALGORITHM

Our fused kernel evaluates eq. (1) one tile of queries at a time. Each thread block owns a $T_M \times D$ tile of Q for a single (batch, head) pair, streams over the $T_N \times D$ tiles of K and V that the causal mask allows it to see, and keeps every score and probability in on-chip storage. This section describes the first correct version of the kernel. The optimizations applied to it afterwards are described in section 6.

2.1. Work Decomposition

The kernel is launched on a two-dimensional grid of $(N/T_M) \times BH$ blocks. Block (i, β) handles query rows iT_M to $(i+1)T_M - 1$ of batch $b = \lfloor \beta / H \rfloor$ and head $h = \beta \bmod H$, so blocks are independent and no two blocks write the same part of O. A block loads its query tile $Q_i \in \mathbb{R}^{T_M \times D}$ once and keeps it on chip for the whole computation. In each loop iteration it loads a key tile directly in transposed form, $K_j^\top \in \mathbb{R}^{D \times T_N}$, by passing a permuted dimension order to cuTile's tile load, together with the matching value tile $V_j \in \mathbb{R}^{T_N \times D}$. The host wrapper checks that N is divisible by both tile sizes, computes the scale $1/\sqrt{D}$ and launches the kernel. By default, $T_M = T_N = 64$.

2.2. Online Softmax

For every query row the block maintains three FP32 quantities: a running maximum m, a running normalization sum ℓ, and an output accumulator $a \in \mathbb{R}^D$, initialized to $m = -\infty$, $\ell = 0$ and $a = 0$. For key/value tile j it computes the score tile $S_j = Q_i K_j^\top / \sqrt{D}$ and then updates the state row by row:

$$\begin{aligned} m^{\text{new}} &= \max\big(m,\ \text{rowmax}(S_j)\big), & \ell &\leftarrow \alpha\,\ell + \text{rowsum}(\tilde{P}_j), \\ \alpha &= \exp(m - m^{\text{new}}), & a &\leftarrow \alpha\, a + \tilde{P}_j V_j, \\ \tilde{P}_j &= \exp\big(S_j - m^{\text{new}}\big), & m &\leftarrow m^{\text{new}}. \end{aligned} \tag{2}$$

After the last tile the block writes $O_i = a/\ell$, cast to FP16.

The recurrence keeps an invariant: after any number of tiles, ℓ and a equal $\sum_k e^{s_k - m}$ and $\sum_k e^{s_k - m} v_k$ over every key k seen so far, where m is the largest score seen so far. When a new tile raises the maximum, multiplying the old totals by $\alpha = e^{m - m^{\text{new}}}$ re-expresses them relative to the new maximum, so the invariant still holds. At the end the common factor e^{-m} cancels in a/ℓ, which is therefore exactly the softmax-weighted sum of eq. (1).

The recurrence is also what makes the kernel numerically stable. Every exponent it evaluates is at most zero, because $S_j - m^{\text{new}} \le 0$ and $m - m^{\text{new}} \le 0$, so exp never overflows, even for score magnitudes at which a direct e^s would exceed the FP32 range. The largest term in each row equals one, so $\ell \ge 1$ and the final division is well conditioned. Masked scores of $-\infty$ contribute $e^{-\infty} = 0$. In the first iteration $\alpha = e^{-\infty} = 0$, which discards the empty initial state. Because key 0 is visible to every query, the first tile always contains at least one unmasked score per row, so m is finite from then on and the undefined expression $-\infty - (-\infty)$ never occurs.

2.3. Causal Tile Skipping and Masking

Query tile i only attends to keys with index at most $(i+1)T_M - 1$, so its loop runs over $\lceil (i+1)T_M / T_N \rceil$ key/value tiles and never loads a tile that lies entirely above the diagonal. Tiles with $j < \lfloor iT_M / T_N \rfloor$ lie entirely below the diagonal and are used without a mask. Only the remaining tiles, which straddle the diagonal, compare column and row indices and set $s = -\infty$ wherever the key index exceeds the query index. With $T_M = T_N$ and $n = N/T_M$ tiles along the sequence, a (batch, head) pair processes $n(n+1)/2$ tile pairs instead of n^2, close to half the work for long sequences, and only n of them, one per query tile, pay the cost

of masking. Figure 2 shows this schedule for $n = 8$.

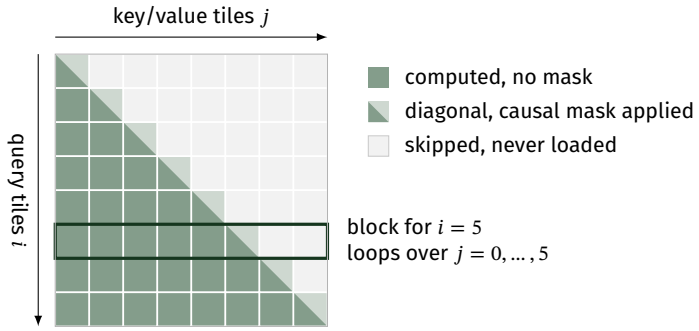


Figure 2. Causal tile schedule for $N = 8T$ with $T_M = T_N = T$, adapted from [8]

2.4. Precision and Memory Traffic

Both matrix products, $Q_i K_j^\top$ and $\tilde{P}_j V_j$, run on tensor cores through cuTile's `mma` with FP16 operands and FP32 accumulation. The score tile, the statistics m and ℓ and the accumulator a all stay in FP32. The only down-conversion inside the loop is the cast of $\tilde{P}_j$, whose entries lie in $[0, 1]$, to FP16 before it is multiplied by V_j.

Each block reads its Q tile once, reads each K and V tile it needs once, and writes its O tile once. The score and probability tiles never leave the chip, and no $N \times N$ tensor is ever allocated in global memory. Because every query tile re-reads the K and V tiles it needs, this traffic still grows with N^2, but it moves only D-wide tiles of K and V rather than the full score and probability matrices.

3. CORRECTNESS

We compare every kernel against an explicit PyTorch implementation of causal attention that upcasts the FP16 inputs to FP32, forms the full score matrix, masks future keys with $-\infty$, applies the softmax and multiplies by V, all in FP32. The runner `run_correctness.py` validates each case's inputs with the same checks as the command-line interface, runs the kernel and the reference, and compares the outputs element by element.

3.1. Pass Criterion and Tolerance

A case passes if the output contains no NaN or infinite values and every element o satisfies, for the reference value r,

$$|o - r| \le \tau_a + \tau_r\,|r|, \qquad \tau_a = \tau_r = 2 \times 10^{-2}. \tag{3}$$

For each case we report the maximum absolute error, the maximum relative error $|o - r| / \max(|r|, 10^{-3})$, and the largest ratio of the error to its bound, which must be at most one.

The relative term is about 40 times the FP16 unit roundoff of $2^{-11} \approx 4.9 \times 10^{-4}$, which leaves room for the rounding of $\tilde{P}_j$ and of the output to FP16. The absolute term covers outputs near zero, which occur when a nearly uniform softmax averages many values of V. Real bugs, such as a wrong mask or a missing rescaling by α, produce errors as large as the outputs themselves and are still caught. Figure 3 shows the resulting pass region.

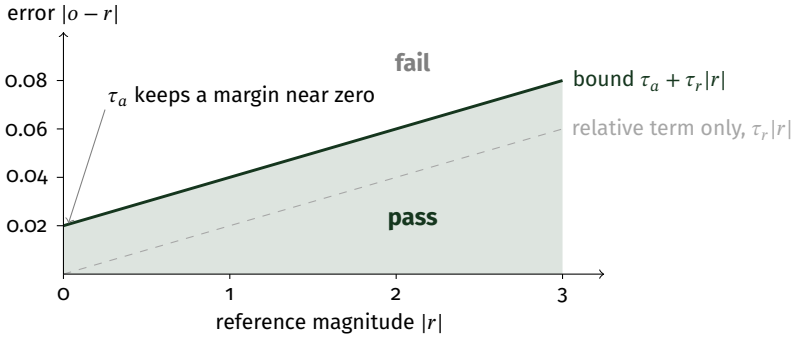


Figure 3. Pass region of the mixed tolerance in eq. (3)

3.2. Test Cases

Table 2 lists the 84 cases. Seventy-two cover every combination of head dimension, sequence length, score magnitude and three (batch, head) shapes, and twelve more use the largest supported shape. The head count $H = 5$ is not a power of two, which checks the block-index arithmetic of section 2. The suite runs with four tile configurations, where T_M is the number of query rows in a Q tile and T_N the number of key rows in a K or V tile. With 128-wide tiles the shortest sequence, $N = 128$, is exactly one tile, so sequence lengths equal to and larger than one tile are both covered.

Table 2. Correctness test grid.

Parameter	Values
Head dimension D	32, 64, 128
Sequence length N	128, 512, 2048, 8192
Batch and heads (B, H)	(1, 1), (2, 8), (3, 5)
Largest shape	$(B, H) = (16, 32)$ with $N \in \{128, 512\}$
Score magnitude	tiny ($\sigma = 0.03$), large ($\sigma = 7$)
Tile sizes $T_M \times T_N$	64×64, 64×128, 128×64, 128×128
Seed	CRC32 checksum of the case identifier
Total	$3 \times 4 \times 2 \times 3 + 2 \times 3 \times 2 = 84$ cases per tile size

Q and K are standard normal values scaled by σ, and V is unscaled, so every score has a standard deviation of σ^2 for any D and the outputs stay of order one. With $\sigma = 0.03$ the softmax is nearly uniform. With $\sigma = 7$ the largest scores reach about 240, and since e^{240} far exceeds the FP32 limit of about $e^{88.7}$, these cases fail unless the maximum is subtracted. Inputs are generated with NumPy from a seed equal to the CRC32 checksum of the case identifier, so every run sees the same inputs. Inputs with an unsupported dtype or shape, or a sequence length not divisible by the tile sizes, are rejected before launch with exit status 2.

All four versions of the fused kernel from section 6 pass all 84 cases with each of the four tile configurations, with no NaN or infinite values in any output. The largest absolute error over all runs is 1.98×10^{-3} and the largest tolerance ratio is 0.032, so every element lies more than 30 times inside its bound. The largest relative error, 0.38, occurs where the reference is close to zero, which is the case the absolute term is meant to handle. The unfused baseline also passes every case, with a largest tolerance ratio of 0.042.

4. BENCHMARK METHODOLOGY

All performance numbers in this report come from one script, `benchmark_forward.py`, which times three implementations of causal attention on identical inputs and records latency, throughput and speedup for every problem size. Table 3 summarizes the setup.

4.1. Implementations Compared

Fused kernel. The kernel of section 2. The script loads `fused_attention` from a kernel file given with `--kernel`, by default `attention/fused/fused.py`. Each version from the optimization history in section 6 lives in its own file under `fused_kernels/`, so every version is measured by the same code.

Unfused baseline. A cuTile implementation that deliberately writes S and P to global memory, split into three kernels on the same $(N/T_M) \times BH$ grid. The first computes the masked, scaled score tiles with `mma` and stores S in FP32. The second reads each row of S twice, once to find the row maximum and sum and once to normalize, and stores P in FP16. The third multiplies P by V with an FP32 accumulator and writes O. Like the fused kernel it skips every tile above the causal diagonal, so the comparison isolates the cost of materializing S and P rather than the cost of computing masked tiles. The $B \times H \times N \times N$ buffers for S and P are allocated inside each call.

PyTorch SDPA. `torch.nn.functional.scaled_dot_product_attention` with `is_causal=True`, called on the same FP16 tensors. PyTorch chooses its own backend for these inputs. SDPA serves as a vendor reference and is not part of the speedup grade.

NVIDIA TileGym. The causal attention kernel `tile_fmha` from NVIDIA's TileGym library [7], which autotunes its own tile configuration. It is timed by `benchmark_tilegym.py` with the same procedure and inputs.

4.2. Problem Sizes

We fix $B = 2$ and $H = 8$ and sweep $D \in \{64, 128\}$ and $N \in \{128, 512, 2048, 8192\}$, giving eight configurations that range from short sequences, where launch overhead and filling the GPU dominate, to long sequences, where the quadratic cost dominates. Q, K and V are drawn from a standard normal distribution in FP16 with a fixed seed. The fused and unfused kernels receive the same tile sizes, set with `--tile-m` and `--tile-n`, and every N is divisible by both 64 and 128.

4.3. Timing Procedure

Each implementation is timed in three stages for every configuration. The first call is timed on the host between two device synchronizations. For the cuTile kernels it includes just-in-time compilation, so it is reported separately as the one-time compilation cost. Ten warm-up iterations follow, and then 30 measured iterations. Each measured iteration is bracketed by a pair of CUDA events on the current stream and followed by a synchronization, and the reported latency is the median of the 30 event times, which keeps an occasional slow iteration from distorting the result. Figure 4 summarizes the procedure.

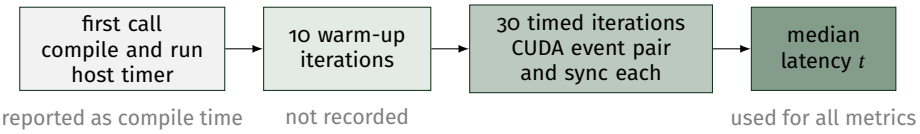


Figure 4. Timing procedure for each implementation and configuration

4.4. Metrics

From the median latency t we compute an effective throughput. Each of the two matrix products costs $2N^2D$ floating-point operations per head, and the causal mask removes about half of them, so

$$\text{throughput} = \frac{2BHN^2D}{t}. \tag{4}$$

The same operation count is used for every implementation, so throughput compares useful work per second. For each configuration we report the speedup of the fused kernel over the unfused baseline and over SDPA as the ratio of their median latencies. Speedups summarized over several configurations use the geometric mean of the per-configuration ratios, so that the longest sequences do not dominate. For every kernel version and tile size the script writes a per-configuration CSV, a summary CSV with averages per N, per D and overall, and the latency, throughput and speedup plots used in section 5.

Table 3. Benchmark setup.

Item	Setting
Implementations	fused cuTile, unfused cuTile (3 kernels), PyTorch SDPA, TileGym (separate run)
Shapes	$B = 2$, $H = 8$, $D \in \{64, 128\}$, $N \in \{128, 512, 2048, 8192\}$
Inputs	standard normal, FP16, fixed seed
Tile sizes	$T_M \times T_N$, same for fused and unfused kernels
Warm-up	first call timed separately as compile time, then 10 iterations
Measurement	30 iterations, CUDA events, median latency
Throughput	$2BHN^2D/t$
Speedup	latency ratio per configuration, geometric mean across configurations
GPU	NVIDIA RTX PRO 6000 Blackwell
Compute capability	12.0
Software	NVIDIA driver 595.91.07, PyTorch 2.14.1 (CUDA 13.0), cuda-tile 1.6.0, Python 3.12.3
Profiler	Nsight Compute 2026.1.1

5. PERFORMANCE RESULTS

This section reports the final kernel, version v3 from section 6, with 64×64 tiles. Table 4 lists every configuration and fig. 5 plots latency, throughput and speedup against N.

5.1. Final Kernel

Across the eight configurations the fused kernel is 4.30 times faster than the unfused baseline and 1.33 times faster than PyTorch SDPA, as geometric means. The gap to the unfused baseline grows with sequence length, from about 2.2 times at $N = 128$ to 10.1 times at $N = 8192$. The baseline moves S and P through global memory, so its throughput stays below 50 TFLOP/s for every N, while the fused kernel keeps improving as N grows and reaches 308 TFLOP/s at $D = 64$ and 341 TFLOP/s at $D = 128$. The fused kernel is ahead of SDPA in every configuration, by 1.08 to 1.99 times, with the largest margin at $N = 2048$. At $N = 128$ every implementation finishes in 12 to 30 microseconds and each head has only two query tiles, so latency there is set by launch overhead and the speedups say little about the kernels themselves. The one-time just-in-time compilation on the first call, about 0.5 to 0.7 s, is excluded from all these numbers.

We also compare against `tile_fmha`, the cuTile attention kernel in NVIDIA's TileGym library [7], timed with the same procedure in a separate run. Our kernel is faster in all eight configurations, by 1.15 to 1.53 times and 1.32 times on average. On this GPU TileGym selects the same 64×64 tiles and occupancy as our kernel.

5.2. Tile Size Selection

The baseline kernel v0 was benchmarked with six tile configurations, and every later version with the four used in section 3. Table 5 shows the mean throughput and the relative latency over the eight configurations. The 64×64 tile is the fastest on average for every version, and for v3 it is the fastest or tied for fastest in all eight configurations, so it is the default. To find out why, we profiled the baseline kernel v0 with Nsight Compute at $D = 64$ and $N = 8192$ for the slowest small tile (32×64), the chosen tile and the slowest large tile (128×128). All three run one 256-thread block per SM with 255 registers per thread, so occupancy is the same and the difference lies elsewhere (fig. 6).

The 32×64 tile executes 65% more instructions than 64×64 (624 M against 379 M), although its tensor-core (HMMA) instruction count is the same, so all the extra instructions are non-matrix work. With half as many query rows per tile, the kernel takes twice as many tile steps, and each step pays the same fixed cost for loading K and V and for loop and address arithmetic. Each K and V tile is also loaded by twice as many blocks, which roughly doubles L2 traffic.

Table 4. Final kernel (v3, 64×64 tiles), $B = 2$, $H = 8$. Speedups are latency ratios, so values above one mean our kernel is faster. The TileGym column comes from a separate run with the same timing procedure.

D	N	Latency (ms)	TFLOP/s	vs unfused	vs SDPA	vs TileGym
64	128	0.0120	2.8	2.34×	1.08×	1.42×
64	512	0.0178	30.2	2.62×	1.24×	1.21×
64	2048	0.0425	202.1	7.64×	1.99×	1.49×
64	8192	0.4457	308.3	13.45×	1.23×	1.15×
128	128	0.0140	4.8	2.08×	1.20×	1.40×
128	512	0.0220	48.8	2.40×	1.31×	1.23×
128	2048	0.0731	235.2	4.84×	1.58×	1.53×
128	8192	0.8066	340.8	7.61×	1.21×	1.19×
Geometric mean				4.30×	1.33×	1.32×

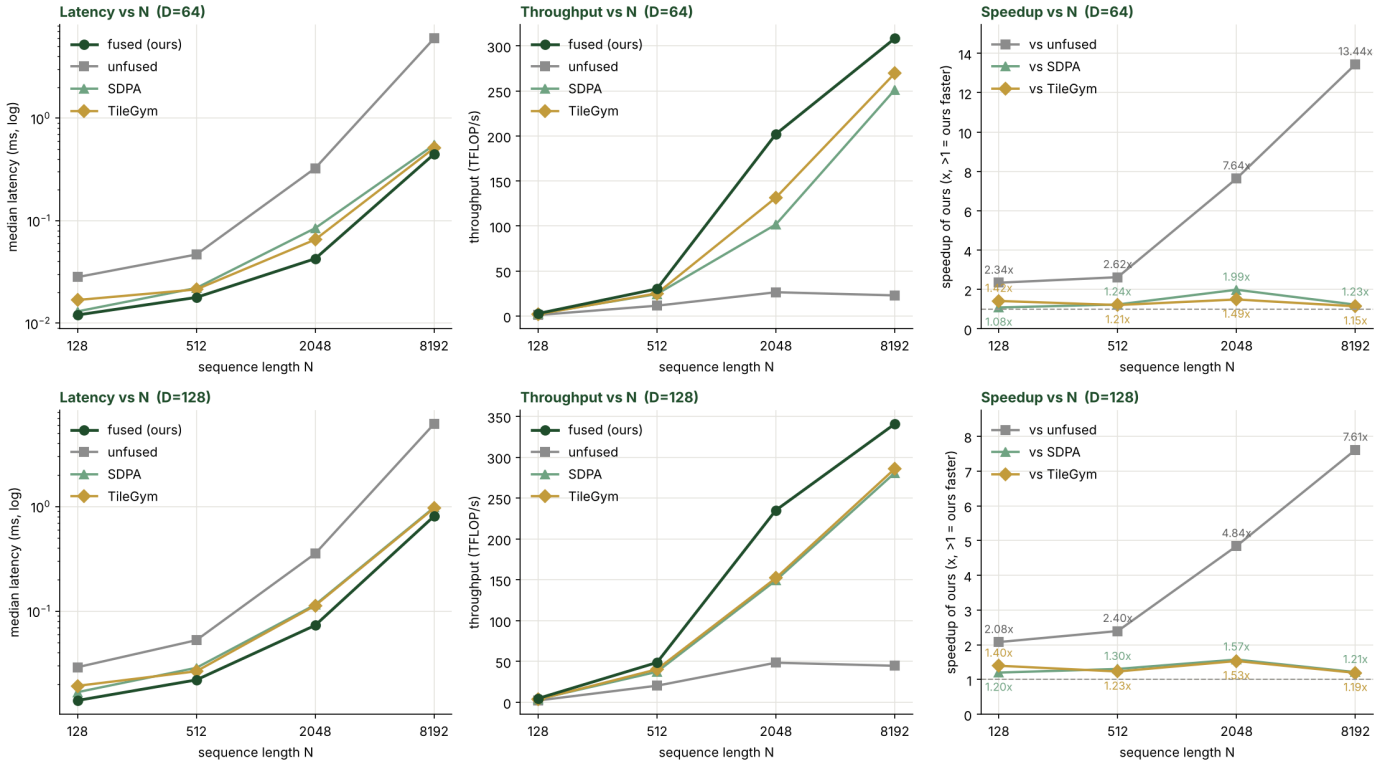


Figure 5. Latency, throughput and speedup of the final kernel with 64×64 tiles against the unfused baseline, PyTorch SDPA and TileGym

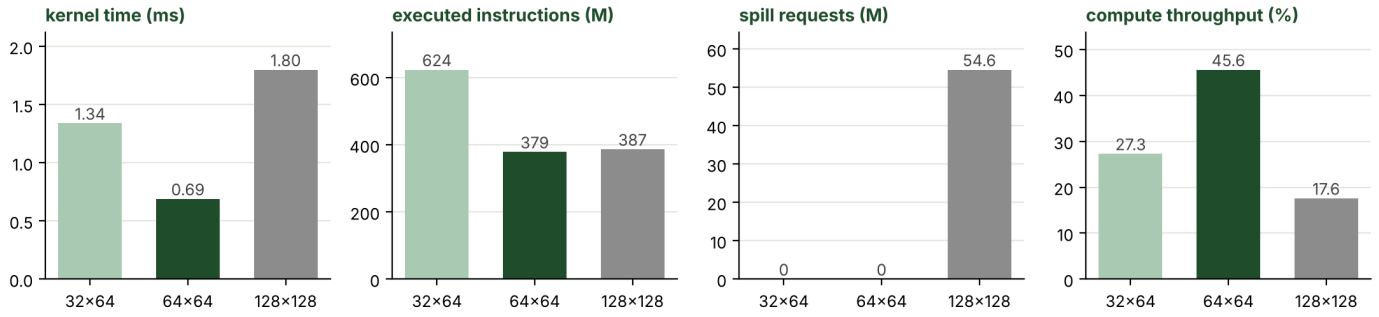


Figure 6. Nsight Compute profile of the baseline kernel at $D = 64$, $N = 8192$ for a small, the chosen and a large tile

The 128×128 tile executes about as many instructions as 64×64, but its data does not fit in the 255 registers available to each thread, so the compiler spills values to local memory. Nsight counts 54.6 M spill requests, which make up 89% of all L2 traffic. Reading a spilled value back from L2 takes hundreds of cycles instead of one, so warps spend most of their time waiting. In 85% of cycles no warp is ready to run, against 64% for 64×64, and compute throughput drops from 46% to 18% of peak.

6. PROFILING AND OPTIMIZATION HISTORY

6.1. Profiling the Baseline

We profiled the baseline kernel v0 with 64×64 tiles at $D = 64$ and $N = 8192$, the configuration where it fell furthest behind SDPA, running at 0.79 times SDPA's speed (0.94 times at $D = 128$, $N = 8192$). Table 6 summarizes what Nsight Compute shows. Nsight's rules estimated three possible gains: **higher occupancy** (54.4%), **better load balance across SMs** (8.9%), since SM

Table 5. Tile size comparison over the eight configurations. Relative latency is the geometric mean of each tile's latency divided by the 64×64 latency, so values above one are slower.

	v0		v3		
Tile $T_M \times T_N$	TFLOP/s	Rel. latency	TFLOP/s	Rel. latency	vs SDPA
32×64	61.0	1.39×	–	–	–
64×32	89.5	1.08×	–	–	–
64×64	99.3	1.00×	146.6	1.00×	1.33×
64×128	93.0	1.04×	118.3	1.19×	1.12×
128×64	51.8	2.14×	67.5	2.44×	0.55×
128×128	27.7	3.30×	26.7	4.27×	0.31×

Table 6. What Nsight Compute shows for the baseline kernel at $D = 64$, $N = 8192$.

Question	Answer
Spilling?	No, 0 spill requests.
Compute- or memory-bound?	Neither. Compute throughput is 45.6% and memory throughput 38.6%, both under 60%. DRAM throughput is only 4.6%, with a 97% L2 hit rate.
What limits it?	**Latency**: in 64% of cycles no warp is ready to issue an instruction.
Why so few ready warps?	**Low occupancy**: 255 registers per thread for 256 threads and 82 KB of shared memory per block allow only one block (8 warps) per SM, which is 2 warps per scheduler out of 12 (16.7% theoretical and 12.3% achieved occupancy).

busy times varied by +9.9% and −7.4% around the mean, and **fusing FP32 instructions** (4.8%), since 107.8 M FP32 instructions were not fused against 74.2 M fused. We made the two cheap changes first and then tackled occupancy, which promised the largest gain.

6.2. Heavy-First Block Order

With causal masking the work per block is uneven. At $N = 8192$ the last query tile of a head processes 128 key/value tiles and the first only one. In v0 the query-tile index varies fastest across the launch, so the heaviest blocks start last and run alone at the end while other SMs sit idle. v1 swaps the grid axes, putting the (batch, head) index on the fast launch axis, and reverses the query-tile index so that the heaviest blocks start first:

```
bh = ct.bid(0)                          # batch*head on the fast launch axis
i  = ct.num_blocks(1) - 1 - ct.bid(1)   # heaviest (last) query tiles first
```

On the profiled configuration the kernel time drops from 690 to 632 μs, an 8.3% reduction close to Nsight's estimate of 8.9%, and the load-imbalance warning disappears.

This is the longest-processing-time-first rule from scheduling theory [1]. The GPU hands blocks to SMs roughly in launch order as slots free up. If the longest jobs start first, the short ones fill the gaps at the end, and the finish time is guaranteed to be within a factor of 4/3 of the best possible. Figure 7 shows the effect for the eight query tiles of one head on three SMs, where heavy-first ordering finishes in 13 time units instead of 15.

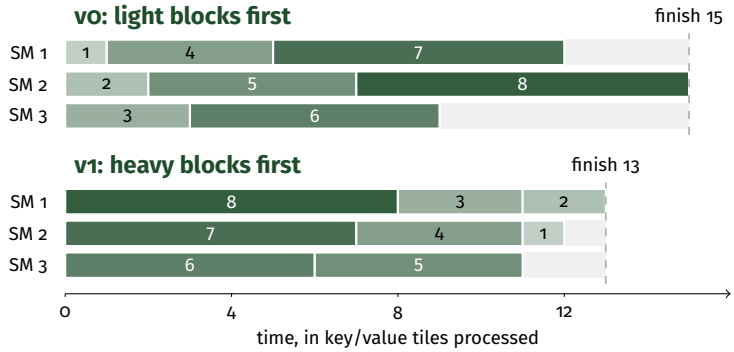


Figure 7. Eight query tiles of one causal head, needing 1 to 8 key/value tiles of work, scheduled greedily on three SMs. Grey marks idle time.

6.3. Base-2 Exponentials and Fast Math

The hardware computes only 2^x, so every `exp` hides an extra multiplication by $\log_2 e$, and scaling the scores by $1/\sqrt{D}$ was another separate multiplication. v2 folds both into one constant $c = \log_2 e/\sqrt{D}$, computed once on the host, so each score needs a single fused multiply-add $s \cdot c - m$ followed by `exp2`. Since $e^x = 2^{x \log_2 e}$ the result is unchanged. v2 also flushes subnormal results of `exp2` to zero and uses an approximate division for the final a/ℓ:

```
scale = LOG2E / math.sqrt(D)                    # host: one folded constant
```

```
p   = ct.exp2(s * scale - m_new, flush_to_zero=True)   # FFMA + EX2 per score
out = ct.truediv(acc, l, flush_to_zero=True, rounding_mode=ct.RoundingMode.APPROX)
```

On the profiled configuration executed instructions fall from 379 M to 255 M, a 33% reduction, and the kernel time drops from 632 to 557 μs, a further 11.9%. The largest error over all correctness cases moves only from 1.95×10^{-3} to 1.98×10^{-3}.

6.4. Two Blocks per SM

Up to this point each SM held one block of 8 warps, which gave each of its four schedulers only 2 warps to choose from, while each warp could issue only about once every 5 cycles. With so few warps the schedulers were idle most of the time. Adding `@ct.kernel(occupancy=2)` asks the compiler to fit two blocks on each SM, which caps registers at 128 per thread and forces smaller shared-memory buffers. Registers fall from 255 to 128 per thread and shared memory from 82 to 25 KB per block, with no spilling. Two blocks now fit on each SM, achieved occupancy rises from 12% to 25%, compute throughput rises from 58% to 76%, and the kernel time drops from 557 to 447 μs, a further 19.8%. Figure 8 summarizes how each step changed the profile.

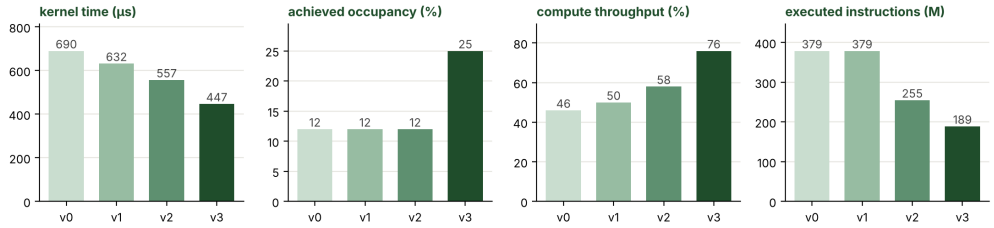


Figure 8. Nsight Compute metrics for each kernel version with 64×64 tiles at $D = 64$, $N = 8192$

6.5. Effect of Each Optimization

Table 7 summarizes every version with 64×64 tiles over the eight benchmark configurations. Each step improves on the previous one on average. The baseline was slower than SDPA in three configurations, most at $N = 8192$ with $D = 64$, while the final kernel is faster than SDPA in all eight, by 1.08 to 1.99 times.

Table 7. Every version with 64×64 tiles. Speedups are geometric means over the eight configurations.

Version	Mean TFLOP/s	vs v0	vs unfused	vs SDPA
v0 baseline	99.3	1.00×	3.25×	1.01×
v1 heavy-first	111.6	1.08×	3.51×	1.09×
v2 exp2 and fast math	123.7	1.20×	3.89×	1.22×
v3 occupancy 2	**146.6**	**1.32×**	**4.30×**	**1.33×**

We tuned the kernel at 64×64 but benchmarked every version with all four tile sizes (fig. 9). Heavy-first ordering and the base-2 exponentials do not depend on the tile size, so they improve every tile, by 1.05 to 1.14 times per step. The occupancy hint is specific to 64×64. It caps registers at 128, which fits a 64×64 tile without spilling, but brings nothing for 64×128 and makes the 128-row tiles slower, because their larger working set spills even more. With all three changes, 64×64 is the fastest or tied for fastest tile in every configuration.

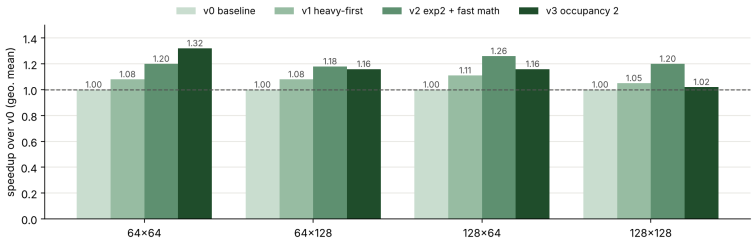


Figure 9. Speedup of each version over the baseline for each tile size, as geometric means over the eight configurations

6.6. Rejected Experiments and Remaining Bottleneck

Table 8 lists five further changes that we tried on top of the two-blocks-per-SM kernel and did not keep. Raising the occupancy hint to three blocks per SM limits each thread to 80 registers. It helps slightly at $D = 64$, but at $D = 128$ the kernel spills 676 K requests to local memory and becomes 40 to 77% slower. Load-latency hints of 4 and 8 on the K and V loads make no measurable difference.

The last two experiments follow NVIDIA's tuned cuTile attention kernel [7, 8]. Splitting the key loop into unmasked and masked parts, the largest single gain in NVIDIA's tuning study, changes nothing for us because the mask branch is already skipped below the diagonal at no cost. Loading V just before $\tilde{P}_j V_j$, as TileGym does, also has no effect.

For the final kernel Nsight still reports 33.3% theoretical occupancy, limited by registers, and estimates 24% more from higher occupancy, but capping registers below 128 causes spilling. Occupancy and spilling now trade off, so further gains need deeper algorithmic or register-allocation changes.

Table 8. Rejected experiments, with latency in ms at $N = 8192$. The kept two-blocks-per-SM kernel is the reference.

Change		$D = 64$	$D = 128$	Verdict
`occupancy=2`	reference	0.447	0.808	kept
`occupancy=3`	80 registers	0.426	1.131	spills, 40–77% slower at $D = 128$
K/V `latency=4`		0.449	0.813	no effect
K/V `latency=8`		0.441	0.814	no effect
Split key loop	unmasked, then masked	0.447	0.813	no effect
Late V load	just before $\tilde{P}_j V_j$	0.448	0.815	no effect

7. ADVANCED TOPICS - GROUPED-QUERY ATTENTION

In grouped-query attention (GQA) the H query heads share a smaller number H_{kv} of key and value heads, with H_{kv} dividing H. Each group of H/H_{kv} consecutive query heads reads the same K and V head, which shrinks the key/value tensors and the KV cache by the same factor. $H_{kv} = H$ is ordinary multi-head attention and $H_{kv} = 1$ is multi-query attention. Figure 10 shows the head mapping for $H = 8$.

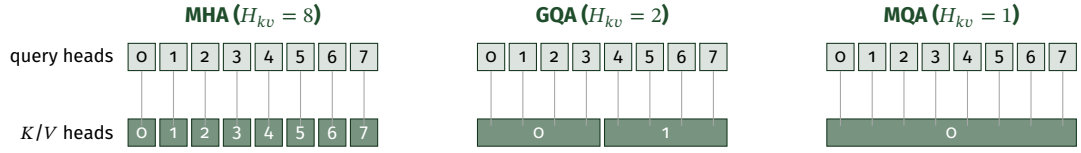


Figure 10. Query heads and the key/value heads they read for $H = 8$. Query head h reads key/value head $\lfloor h/(H/H_{kv}) \rfloor$.

7.1. Kernel and Validation

The fused GQA kernel is the final kernel of section 6 with one change. The grid still has one block per query tile and query head, and each block computes its key/value head as

```
kv_h = h // QUERY_GROUP        # QUERY_GROUP = H // H_kv
```

and loads its K and V tiles from head `kv_h` instead of `h`. K and V are never expanded in memory, and the output keeps all H query heads. The unfused baseline makes the same change in its score and PV kernels. The validation code, the command-line interface `run_attention_gqa.py` and the reference follow the MHA versions, with two extra checks: K and V must have the same head count, and H_{kv} must divide H. The FP32 reference expands K and V with `repeat_interleave` so that query head h sees key/value head $\lfloor h/(H/H_{kv}) \rfloor$.

7.2. Correctness

The GQA suite has 96 cases, covering $D \in \{32, 64, 128\}$, $N \in \{128, 512, 2048, 8192\}$, both score magnitudes and $H_{kv} \in \{1, 2, 4, 8\}$ with $B = 4$ and $H = 8$. It uses the same seeds, tolerance and NaN check as section 3, passes K and V to the kernel with only H_{kv} heads, and processes the reference in whole query groups. The fused kernel passes all 96 cases with each of the four tile configurations, with a largest absolute error of 1.98×10^{-3} and a largest tolerance ratio of 0.034. The unfused GQA baseline also passes every case.

7.3. Performance

We benchmark with $B = 2$, $H = 8$, $H_{kv} \in \{1, 2, 4, 8\}$, $D \in \{32, 64, 128\}$ and $N \in \{128, 512, 2048, 8192\}$ using the procedure of section 4, with SDPA called with `enable_gqa=True`. The operation count depends on H but not on H_{kv}, so throughput is computed as for MHA. Table 9 shows that the kernel's speed does not depend on H_{kv}: with 64×64 tiles it averages about 131 TFLOP/s for every head count and is 1.27 to 1.30 times faster than SDPA and 4.7 to 4.8 times faster than the unfused baseline. Sharing K and V reduces memory traffic, but the MHA kernel was already latency-bound with a 97% L2 hit rate, so fewer distinct K and V tiles bring no measurable gain. An Nsight Compute profile at $H_{kv} = 2$, $D = 64$, $N = 8192$ confirms this. The kernel takes 444.6 μs, uses 128 registers per thread with no spilling, reaches 24.8% achieved occupancy and 75.4% compute throughput, and has a 98.1% L2 hit rate, almost exactly the profile of the MHA kernel.

Table 9. Fused GQA kernel with 64×64 tiles. Speedups are geometric means over the twelve configurations.

H_{kv}	Mean latency (ms)	Mean TFLOP/s	vs unfused	vs SDPA
1 (MQA)	0.147	130.3	4.72×	1.27×
2	0.147	131.2	4.71×	1.27×
4	0.147	131.4	4.80×	1.30×
8 (MHA)	0.148	131.0	4.79×	1.29×

7.4. Rejected Optimizations

We tried two GQA-specific changes at $H_{kv} = 2$. A latency hint of 4 on the K and V loads changes latency by less than 6%, mostly for the worse. Head packing lets one block process all H/H_{kv} query heads of a group, so each K and V tile is loaded once per

group instead of once per query head. With 32×64 tiles a block then holds 128 query rows, and at $D = 128$ the kernel becomes 3.4 to 4.9 times slower than SDPA, most likely the same register spilling measured for 128-row tiles in section 5. Lowering the occupancy hint to one block per SM only reduces this to 2.9 to 4.3 times. With 16×64 tiles a block holds 64 rows again and matches the unpacked kernel, so head packing gives no benefit, and the final GQA kernel keeps one query head per block.

8. ADVANCED TOPICS - BACKWARD PASS

Training needs the gradients of a loss $\mathcal{L}$ with respect to Q, K and V. Differentiating eq. (1) directly needs the probability matrix P, which is $N \times N$ and is exactly what the fused forward pass avoids storing. We follow the recompute approach of FlashAttention-2 [5]: the forward pass saves one number per query row, and the backward pass rebuilds each tile of P on chip when it needs it.

8.1. Gradients

With $S = QK^\top/\sqrt{D}$, $P = \mathrm{softmax}(S)$ under the causal mask, $O = PV$ and the upstream gradient $dO = \partial\mathcal{L}/\partial O$, the chain rule gives

$$\begin{aligned} dV &= P^\top dO, & dS_{ij} &= P_{ij}\left(dP_{ij} - \Delta_i\right), \\ dP &= dO\,V^\top, & dQ &= dS\,K/\sqrt{D}, \\ \Delta_i &= \textstyle\sum_d dO_{id}\,O_{id}, & dK &= dS^\top Q/\sqrt{D}. \end{aligned} \tag{5}$$

The softmax gradient needs the row sums $\Delta_i = \sum_j P_{ij}\,dP_{ij}$. Substituting $dP = dO\,V^\top$ and $O = PV$ gives $\sum_j P_{ij} \sum_d dO_{id} V_{jd} = \sum_d dO_{id} O_{id}$, so Δ needs only O and dO, two $N \times D$ tensors, and not P.

To rebuild P, the forward pass stores the final statistics of eq. (2) for every query row as one log-sum-exp value, $L_i = m_i + \log_2 \ell_i$, in the base-2, pre-scaled units of the final kernel (section 6). Any tile of P then follows from one matrix product and one exponential, $P_{ij} = 2^{c\,Q_i K_j^\top - L_i}$ with $c = \log_2 e/\sqrt{D}$, with no running maximum and no rescaling. The extra storage is one FP32 value per query row, BHN values in total, which is $2/D$ of the size of O.

8.2. Kernels

The implementation, `attention/backward/fused_backward.py`, has three steps:

1. **Forward.** The final kernel of section 6, which additionally writes m and ℓ for every row. The host forms L.
2. **Preprocess.** $\Delta = \mathrm{rowsum}(dO \circ O)$, computed in PyTorch. It is a single memory-bound pass over two $N \times D$ tensors and is needed by both gradient kernels.
3. **Two gradient kernels.** One produces dK and dV, the other dQ.

The dK, dV **kernel** launches one block per key/value tile j and (batch, head) pair. The block keeps K_j, V_j and FP32 accumulators for dK_j and dV_j on chip, and loops over the query tiles i that can see tile j, from the diagonal tile to the last one. Loading Q_i and dO_i a second time in transposed form, with the same permuted dimension order as K in the forward pass, gives the transposed scores $S^\top = K_j Q_i^\top$ directly, so no tile is ever transposed in registers:

```
st  = ct.mma(k, qt, st)                          # S^T  = K Q^T
pt  = ct.exp2(st * scale - lse)                  # P^T, rebuilt from L
dv  = ct.mma(pt.astype(np.float16), do, dv)      # dV  += P^T dO
dpt = ct.mma(v, dot, dpt)                        # dP^T = V dO^T
dk  = ct.mma((pt * (dpt - delta)).astype(np.float16), q, dk)   # dK += dS^T Q
```

The dQ **kernel** launches one block per query tile i, in the same heavy-first order as the forward pass. It keeps Q_i, dO_i, L_i and Δ_i on chip and loops over the visible key/value tiles as the forward pass does, accumulating $dQ_i \mathrel{+}= dS\,K_j$ with $dS = P \circ (dP - \Delta)$. Both kernels skip tiles above the diagonal and mask only the diagonal tiles, exactly as in section 2, and both scale their result by $1/\sqrt{D}$ once, at the end.

Each block accumulates its output tile on chip and writes it once. In exchange, both kernels rebuild P and dP: every visible (query, key) tile pair costs seven tile matrix products ($S^\top$, dV, $dP^\top$ and dK in one kernel, S, dP and dQ in the other) instead of the minimum of five.

Autograd. A `torch.autograd.Function` runs the forward kernel in `forward` and saves (Q, K, V, O, L). Its `backward` computes Δ and launches the two gradient kernels. Users call `fused_attention_autograd(q, k, v)`, and `o.backward(dO)` fills `q.grad`, `k.grad` and `v.grad`. Only $O(BHND)$ memory is saved between the forward and backward passes, against the $O(BHN^2)$ probability matrix that autograd keeps for unfused attention.

8.3. Correctness

We compare O, dQ, dK and dV with a naive PyTorch reference in FP32. The reference upcasts the same FP16 inputs to FP32 and writes attention directly with PyTorch operations: it forms the full score matrix $S = QK^\top/\sqrt{D}$ with one matrix product,

sets the entries above the diagonal to $-\infty$, applies a row-wise softmax and multiplies by V. Its gradients are computed by autograd with the same dO. A case passes when every output is finite and its relative error $\|x - x_{\text{ref}}\|/\|x_{\text{ref}}\|$ is at most 2×10^{-2}. The seven cases cover both head dimensions and sequence lengths from 128 to 2048. Q, K and V are drawn from a normal distribution with standard deviation $\sigma \in \{1, 3, 7\}$, so a larger σ gives larger scores and a sharper softmax. The upstream gradient dO is standard normal, and each case uses a fixed seed. All seven pass with a margin of at least 20×.

8.4. Performance

We time forward plus backward with $B = 2$, $H = 8$, $D \in \{64, 128\}$ and $N \in \{128, 512, 2048, 8192\}$, taking the median of 20 runs (`benchmarks/benchmark_backward.py`), and record the peak GPU memory allocated beyond the inputs. We compare against two baselines. The first is **naive PyTorch**, the attention code commonly used in training. It follows the same steps as the correctness reference but keeps the matrix products in FP16, with only the softmax in FP32. Autograd computes its backward pass and stores the full $N \times N$ probability matrix for it. The unfused cuTile kernels of section 4 have no backward pass, so this PyTorch version serves as the unfused baseline here. The second is **PyTorch SDPA** with its built-in fused backward. Throughput counts the backward pass as 2.5× the forward FLOPs, the usual FlashAttention convention, and does not count recomputation.

Table 10. Forward + backward with 64 × 64 tiles: median time, throughput and peak extra memory. Speedup = other / fused.

		Time (ms)			Fused speedup			Memory (MB)	
D	N	Fused	Naive	SDPA	vs naive	vs SDPA	TFLOP/s	Fused	Naive
64	128	0.142	0.307	0.087	2.16×	0.61×	0.8	1.8	4.5
64	512	0.142	0.305	0.096	2.14×	0.67×	13.2	7.0	66.2
64	2048	0.247	5.208	0.241	21.10×	0.97×	121.8	28.1	1036
64	8192	2.348	86.297	1.860	36.76×	0.79×	204.9	112.5	16480
128	128	0.143	0.298	0.088	2.08×	0.62×	1.6	3.5	5.0
128	512	0.164	0.304	0.096	1.86×	0.58×	22.9	14.0	68.2
128	2048	0.537	5.348	0.357	9.96×	0.67×	112.0	56.1	1044
128	8192	6.073	86.598	3.249	14.26×	0.54×	158.4	224.5	16512

Table 10 shows three regimes.

- **Against naive PyTorch**, fusion pays off as N grows. The fused implementation is 10 to 37× faster at $N \geq 2048$, and at $N = 8192$ it needs 74 to 147× less memory: 112 MB instead of 16.5 GB at $D = 64$. Naive memory grows by about 16× for every 4× in N, ours by 4×.
- **Against SDPA**, the fused implementation reaches 54% to 97% of its forward-plus-backward speed (205 against 259 TFLOP/s at $D = 64$, $N = 8192$) and uses slightly less memory (112.5 against 129.0 MB). Our backward pass takes about four times as long as our forward pass, against the 2.5 to 3 times typical of FlashAttention-2. We expect the main causes to be the seven instead of five tile matrix products, the two FP32 accumulators the dK, dV kernel keeps at $D = 128$, where the gap to SDPA is largest, and the extra transposed loads of Q and dO. Unlike the forward kernel, the backward kernels have not been through the profiling iterations of section 6.
- **At small** N the time stays at about 0.14 ms. It is set by launching three kernels, a few small PyTorch operations (Δ, L and the allocations) and autograd overhead, not by GPU work.

9. LIMITATIONS AND FUTURE WORK

The kernels support only causal attention on FP16 inputs, with $D \in \{32, 64, 128\}$ and a sequence length divisible by the tile sizes. All results come from one GPU, the RTX PRO 6000 Blackwell, and the 64 × 64 tile choice is tuned only for it. The final forward kernel is still limited by registers at 33% occupancy. Larger tiles or more blocks per SM cause register spilling, so the fused kernel cannot gain more from occupancy without deeper changes to how it uses registers. The GQA kernel still processes one query head per block, so query heads that share a key and value head each load it separately and rely on the cache for reuse. Packing the heads of a group into one block made the kernel slower because of the larger tiles. The backward pass is correct and uses far less memory than naive PyTorch, but it reaches only 54% to 97% of the speed of SDPA, and it takes about four times as long as our forward pass.

Masking the last partial tile would allow any sequence length, and BF16 support would cover most training code. Sharing key and value tiles across the query heads of a group without enlarging the tiles could make GQA faster. Running the benchmarks on other GPUs, such as Hopper, would show whether the tile choice and the optimisations carry over.

10. INDIVIDUAL CONTRIBUTIONS

The work was divided as planned in our project plan.

Nishitha D. The FP32 PyTorch reference for causal attention; the unfused cuTile baseline (masked $QK^\top$, softmax and PV kernels); the first Nsight Compute profiles and the identification of the limiting factor; and the backward pass: the forward with the saved log-sum-exp, the dK, dV and dQ kernels, the PyTorch autograd integration, the gradient tests, and the benchmark against naive PyTorch and SDPA.

Harshith M R. The `run_attention.py` command-line interface and input validation; the correctness tests (84 cases, fixed seeds, NaN/Inf check, mixed tolerance, rejection of invalid inputs); the forward benchmark harness (unfused, fused and SDPA, warm-up, median of 30 runs, CSV output); the initial tile-configuration sweep; and grouped-query attention: the GQA reference, unfused baseline and validation, the 96-case tests, the fused GQA kernel, and its benchmark, profiling and tile sweep.

Both. Environment setup; the tiled algorithm and the first fused kernel (scores, scaling, causal masking, online softmax); the profiler-guided optimizations v1–v3, the rejected experiments, the final tile choice and the analysis of the remaining bottleneck; and the final benchmarks and profiles, repository cleanup, report and presentation.

REFERENCES

[1] R. L. Graham, "Bounds on multiprocessing timing anomalies", *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.

[2] A. Vaswani et al., "Attention is all you need", in *Advances in Neural Information Processing Systems*, 2017.

[3] M. Milakov and N. Gimelshein, *Online normalizer calculation for softmax*, 2018. arXiv: 1805.02867.

[4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness", in *Advances in Neural Information Processing Systems*, 2022.

[5] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning", in *International Conference on Learning Representations*, 2024.

[6] NVIDIA, *cuTile Python documentation*, https://docs.nvidia.com/cuda/cutile-python/, 2025.

[7] NVIDIA, *TileGym: cuTile kernel library*, https://github.com/NVIDIA/tilegym, 2025.

[8] A. Morari, A. Zhao, I. Yin, and V. Mehta, *Tuning flash attention for peak performance in NVIDIA CUDA tile*, NVIDIA Technical Blog, https://developer.nvidia.com/blog/tuning-flash-attention-for-peak-performance-in-nvidia-cuda-tile, 2026.